FRIEDRICH-SCHILLER-
UNIVERSITÄT
JENA

# General Training for Programming Competitions & Coding Interviews


December 5, 2024

Winter semester 24/25

Markus Fischer, Paul G. Rump, Christoph Staudt

# Second Part of the Lecture

- In the second part of the lecture, we will discuss *algorithms*.
- The aim is to *give a general idea* of how these algorithms work so you can implement them (and modify if required).
- We *will not*:
  - discuss all the details of the algorithms or prove correctness or asymptotic runtimes in this lecture – this is not a lecture about algorithmics.
  - We will also not discuss how to implement these algorithms in the most efficient way – they use the same basic data structures that we already discussed in the previous weeks.

# Graph Basics

# Definition: Graph

A *Graph* $G$ is a tuple of *vertices* $V$ (also called *nodes*) and *edges* $E$.
The edges are either

- unordered pairs of vertices, for example $\{v_1, v_2\}$. In this case, the graph is called *undirected*.
- Or ordered tuples of vertices, e. g. $(v_1, v_2)$. In this case, the graph is called *directed*.
- Often a (undirected) edge $\{u, v\}$ is written as $uv$.

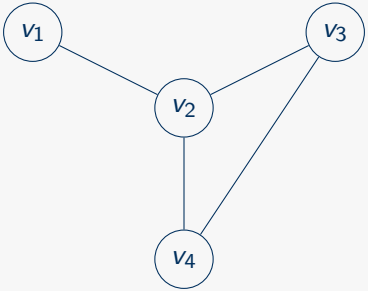
# Definition: Graph

A *Graph* $G$ is a tuple of *vertices* $V$ (also called *nodes*) and *edges* $E$.
The edges are either

- unordered pairs of vertices, for example $\{v_1, v_2\}$. In this case, the graph is called *undirected*.
- Or ordered tuples of vertices, e. g. $(v_1, v_2)$. In this case, the graph is called *directed*.
- Often a (undirected) edge $\{u, v\}$ is written as $uv$.

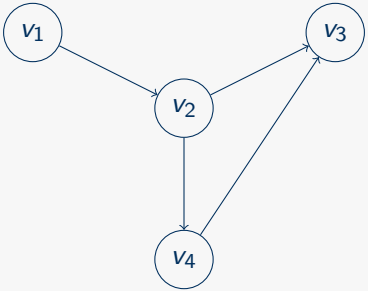
Given a node $v \in V$, the *neighbors* of $v$ is the set of nodes $v'$ that are connected to $v$ via an edge (i. e. $\exists \{v, v'\} \in E$). $v, v'$ are also said to be *adjacent*.
The *degree* of $v$ is the number of neighbors of $v$.
A *clique* is a set of nodes where all nodes of the set are pairwise adjacent.

# Example: Graph



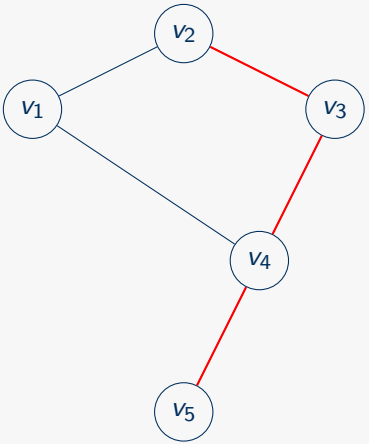

Graph $G = (V, E)$ with $V = \{v_1, v_2, v_3, v_4\}$ and $E = \{\{v_1, v_2\}, \{v_2, v_3\}, \{v_2, v_4\}, \{v_3, v_4\}\}$.
Degree of $v_2$ is 3.
The nodes $v_2, v_3, v_4$ form a clique.

# Example: Directed Graph

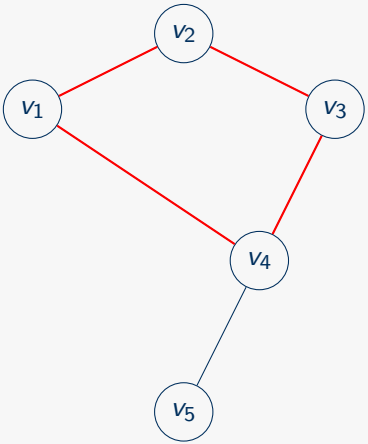



Graph $G = (V, E)$ with $V = \{v_1, v_2, v_3, v_4\}$ and $E = \{(v_1, v_2), (v_2, v_3), (v_2, v_4), (v_4, v_3)\}$.

# Example: Path



# Example: Cycle



# Examples of Special Types of Graphs

- Regular graphs (all nodes have the same degree)
- Complete graphs (all nodes of the graph form a single clique)
- Trees & Forests (there are no cliques of size $> 2$)
- Directed Acyclic Graphs (DAGs)

# General idea: graphs as generalizations of lists

We have seen two basic data structures so far:

- Lists, arrays, stacks, etc. They have a defined linear order, each element has at most one predecessor and at most one successor.
- Graphs and trees: every element can have arbitrary many pre- and successors; in the case of undirected graphs, these are simply neighbors.

*Graphs are a generalization of lists!*

# Basics about graph data representation

There are two basic ways of representing a graph in a data structure:

- as adjacency list, this is in general the best option for large graphs or sparse graphs,
- as adjacency matrix, this is a useful option for small graphs or dense graphs.

Which option you should choose depends very much of the size of the graph and the algorithms you want to use.
Note that in general, you *don't have to store the list of nodes*, as long as you don't have additional parameters on the nodes.

# Example: adjacency list

An adjacency list is a two-dimensional array that stores a list of all neighbors for each node.

- Very memory-efficient, because you only store information about existing edges.
- However, it can take $\mathcal{O}(\log |V|)$ time to check if two nodes $v$,$w$ are adjacent (binary search in the adjacency list of $v$ for node $w$).

# Example: adjacency matrix

An adjacency matrix is a matrix that contains at position $i, j$ a 1 (or some other non-zero) if there is an edge from $v_i$ to $v_j$ in the graph; and 0 otherwise.

- This is very easy to implement and to use.
- However, you will *always* need $|V|^2$ space to store an adjacency matrix. This is very bad for huge graphs, especially when there are only few edges.
- But you can store an additional parameter of the edge (weight, length, capacity, etc) directly in the matrix, no need for an additional data structure.
- Thus it takes $\mathcal{O}(1)$ time to check whether two nodes are adjacent and also get the parameter of the edge.

# Graph Traversion

# Applications of Graph Traversals

- Connectivity check
- Find cycles
- Visit all nodes in the graph to gather information.
- In general: basic building block of *every* graph algorithm.

# Graph Traversion Algorithms

In general, we need to ensure that we visit every node in the graph; and idealy, we visit every node exactly once. To this end, we need to store information about the nodes that have been visited and which to visit next. There are *two different approaches*:

- *Breadth-first-search*: in each iteration, get the next node to visit, and put all its unvisited neighbors into the queue of nodes to visit.
- *Depth-first-search*: follow an arbitrary path of unvisited nodes through the graph; in the end backtrack the path until an unvisited neighbor ist found and start a new path.

In both cases we have to remember which nodes we have already seen. Both have complexity $\mathcal{O}(|V| + |E|)$.
Breadth-first-search can even be used for shortest paths if all edges have the same length.

# Basic code for BFS (adjacency matrix)

./code/bfs.cpp

```cpp
void breadth_first_search_matrix(const IncidenceMatrix& matrix, int n, int start_node, std::function<bool(int)> callback)
{
    // Initialize a empty queue of nodes to visit
    std::queue<int> queue;
    // Initialize a vector that tracks which nodes where already visited
    std::vector<int> visited_nodes(n, 0);
    // Push our start node into the queue
    queue.push(start_node);
    visited_nodes[start_node] = 1;

    // Go through the graph until the queue is empty or we fulfilled the break condition
    int current_node;
    bool found = false;
    while (!queue.empty() && !found)
    {
        // pop the node at front of the queue
        current_node = queue.front();
        queue.pop();

        //Add all unvisited neighbours to the queue and mark them als visited
        for (int i = 0; i < n; ++i)
        {
            if (matrix[n * current_node + i] > 0)
            {
                if (visited_nodes[i] == 0)
                {
                    queue.push(i);
                    visited_nodes[i] = 1;
                }
            }
        }
        // Do something with the node
        found = callback(current_node);
    }
}
```

# Basic code for BFS (adjacency list)

./code/bfs.cpp

```cpp
void breadth_first_search_list(const IncidenceList& list, int start_node, std::function<bool(int)> callback)
{
    // Initialize a empty queue of nodes to visit
    int n = list.size();
    std::queue<int> queue;
    // Initialize a vector that tracks which nodes where already visited
    std::vector<int> visited_nodes(n, 0);
    // Push our start node into the queue
    queue.push(start_node);
    visited_nodes[start_node] = 1;

    // Go through the graph until the queue is empty or we fulfilled the break condition
    int current_node;
    bool found = false;
    while (!queue.empty() && !found)
    {
        // pop the node at front of the queue
        current_node = queue.front();
        queue.pop();

        //Add all unvisited neighbours to the queue and mark them als visited
        for (int neighbour_node : list[current_node])
        {
            if (visited_nodes[neighbour_node] == 0)
            {
                queue.push(neighbour_node);
                visited_nodes[neighbour_node] = 1;
            }
        }
        // Do something with the node
        found = callback(current_node);
    }
}
```

# Shortest Paths

# Shortest Paths

Task:

- Given a graph $G = (V,E)$ with edge-weights $w : E \to \mathbb{R}$, and
- a pair of nodes $s,t \in V$ (called *source* and *target*).
- Find a path $(e_1, e_2, \ldots, e_k)$ that connects $s$ and $t$ such that the sum of the weights of the edges of the path is minimal for all paths that connect $s$ and $t$.

*Many problems* can be reduced to shortest paths.

Different flavors:

- Some algorithms can't handle negative edge weights (note that all algorithms require that there are *no cycles with negative lengths*).
- One differentiates between *SPSP* (Single-Pair-Shortest-Path) and *APSP* (All-Pairs-Shortest-Paths) -problems.
- Directed edges vs. undirected edges does not matter in general.

# SPSP - Dijkstra's algorithm

- Finds shortest paths from a single starting node to all other nodes in the graph (SPSP).
- Basic idea: Build a min-priority-queue that contains unvisited nodes, sorted by their total distance to $s$. This queue is initialized with the pair $(0,s)$, all other nodes have distance $\infty$. In each iteration, the unvisited node which is closest to $s$ is taken from the queue and the distances of its neighbors are updated.
- Requires that all edges have nonnegative weight.
- Time complexity is $\mathcal{O}(|V| + |E| \log |V|)$ using a simple priority queue; in theory $\mathcal{O}(|V| \log |V| + |E|)$ using a Fibonacci heap.

# Dijkstra's algorithm

dijkstra.cpp

./code/dijkstra.cpp

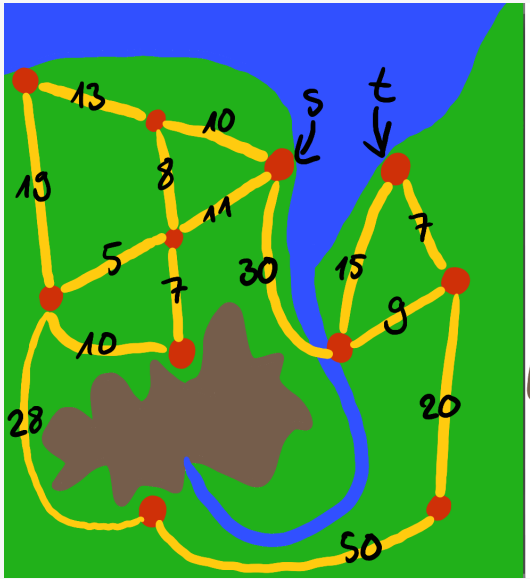
```cpp
int dijkstra(IncidenceMatrix E, int n, int s, int t){
    // initialize all distances as -1:
    std::vector<int> distance(n, -1);
    // initialize a priority-queue<node> that sorts its elements in increasing order:
    std::priority_queue<node, std::vector<node>, std::greater<>> q;
    // add node s with distance 0 to priority queue:
    q.push(node(0, s));

    node current_node;
    while (!q.empty()){
        // get first element from queue and remove it from queue:
        current_node = q.top();
        q.pop();
        // set the distance for the current node:
        if (distance[current_node.second] < 0) {
            distance[current_node.second] = current_node.first;
        }
        // update the distances of all neighbors of the current node to the queue:
        for (int i=0; i < n; i++) {
            if (E[current_node.second * n + i] > 0) {
                if (distance[i] < 0) {
                    // compute distance to node i via current node:
                    int dist_to_neighbor = current_node.first + E[current_node.second * n + i];
                    q.push(node(dist_to_neighbor, i));
                }
            }
        }
    }
    return distance[t];
}
```

# Heuristic adaption of Dijkstra: the A*-algorithm

The A*-algorithm is a well-known *heuristic adaption* of Dijkstra's algorithm. We will discuss it as an example for heuristic approaches.

- Sometimes we have some additional information that is not encoded in the graph.
- For example, in the shortest path setting, our graph might be embedded on a 2-dimensional surface. In this case, we can guess/estimate the distance between any two nodes of the graph without even visiting them.
- This estimate can be used when sorting the priority queue. When deciding which node to visit next, we do not simply choose the node with the shortest distance to the node $s$, but we also take the (guessed) remaining distance to $t$ into account.
- For this to work correctly, it is important that our guess *never over estimates* the remaining distance!

# A* algorithm: example




An example for A*

Land

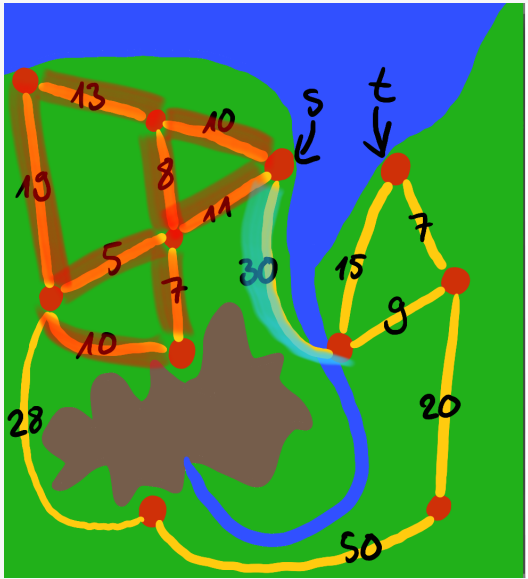
Water

Mountain

City (node)

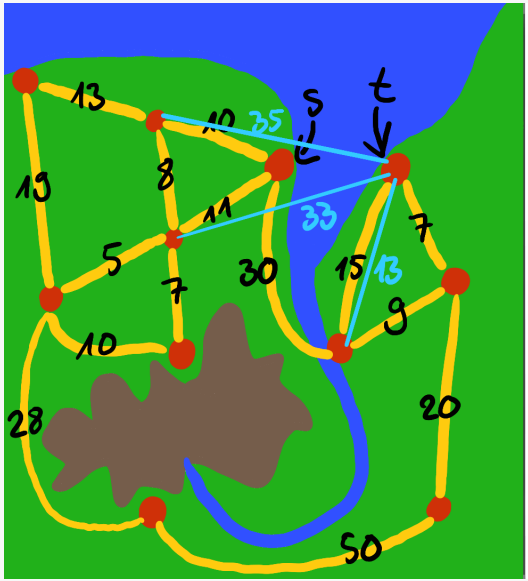
Street (edge)

# A* algorithm: example




Dijkstra:
explores all nodes in the top left before crossing the river.
Also goes to the city in bottom left (distance 44) before reaching t !

# A* algorithm: example




A*-algorithm:
sorts priority queue by
total-distance-to-s
+
estimated-distance-to-t
→goes directly across
the river!

# APSP with dynamic programming

- Using Dijkstra's algorithm to compute the shortest paths between every pair of nodes in a graph would require $\mathcal{O}(|V|(|V| + |E| \log |V|))$ time. In a dense graph with $|E| \in \mathcal{O}(|V|^2)$, the total runtime is $\mathcal{O}(|V|^3 \log |V|)$.
- However, this approach computes lots of sub-paths multiple times. The algorithm of Floyd and Warshall makes use of *dynamic programming* to compute all shortest paths in $\mathcal{O}(|V|^3)$ time.
- Basic idea of Floyd-Warshall: Iterate from 1 to $|V|$, in iteration $i$ compute shortest paths that only use intermediate nodes $\{v_1, \ldots, v_i\}$.
- Floyd-Warshall can handle edges with negative weight.
- Super-easy to implement.

# Floyd-Warshall algorithm

floyd-warshall.cpp

./code/floyd–warshall.cpp

```cpp
std::vector<double> floyd_warshall(IncidenceMatrix E, n){

    // initialize all distances by edges, set distances to self to 0:
    std::vector<double> distances(n*n, std::numeric_limits<double>::infinity());
    for (int i=0; i<n; i++)
        distances[i*n + i] = 0;

    // run Floyd-Warshall:
    for (int k=0; k<n; k++) {
        for (int i=0; i<n; i++) {
            for (int j=0; j<n; j++) {
                // update distance of i,j by path via k:
                distances[i*n + j] = min(distances[i*n + j], distances[i*n + k]+distances[k*n + j]);
            }
        }
    }

    return distances;
}
```

# Exam Questions

# Exam Questions

- If you need an efficient algorithm to find the shortest path between two nodes in a graph where all edges have length 1, which algorithm would you use and why?
- Assume that you need to find a shortest path in a relatively small graph with arbitrary edge lengths, and you have to have a working algorithm very soon. Which algorithm would you choose and why?
- Why does Dijkstra's algorithm not work with negative edge lengths? Give a small example.
- Present an extension for Dijkstra's algorithm (pseudo-code or description) that also returns a list of nodes that form a shortest path. Explain (proof sketch) why that algorithm is correct.
- Explain the difference between an adjacency matrix and an adjacency list? What is the advantage/disadvantage of them and when should each representation be used?